Recap

Trees. Binary trees

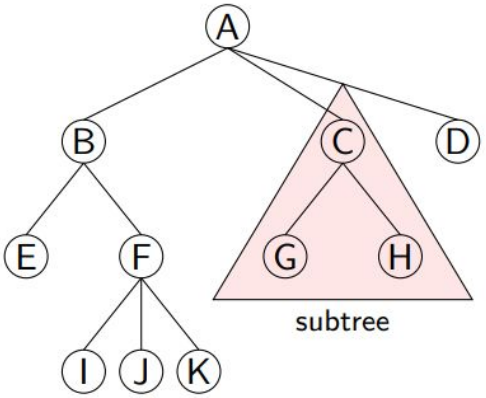
Mădălina Răschip, Cristian Gațu

Faculty of Computer Science
"Alexandru Ioan Cuza" University of Iași, Romania

DS 2020/2021

Trees: terminology

- root: node without parent.
- internal node: has at least one child.
- external node (leaf): node with no children.
- descendants of a node: children, grand children, etc.
- siblings: all other nodes having the same parent.
- subtree: some node and all its descendants.

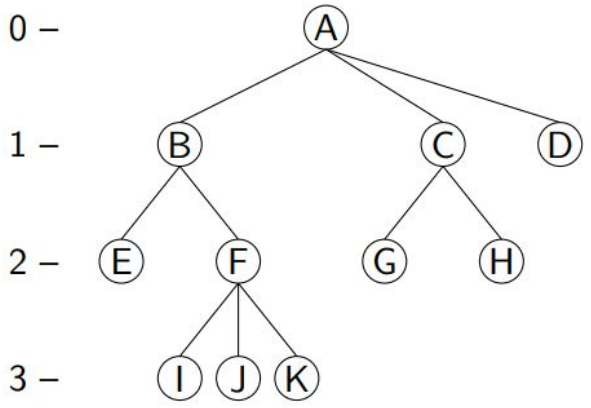


Trees: terminology

- Depth of some node x:

$$\text{depth}(x) = \begin{cases} 0, & x \text{ is the root,} \\ 1 + \text{depth}(\text{parent}(x)), & \text{otherwise.} \end{cases}$$

- tree height: maximum depth of tree nodes.
- height of node x: distance from x to its most far descendant.



Content

Trees

Binary tree (`BinTree`)

Application: integer expression representation as trees

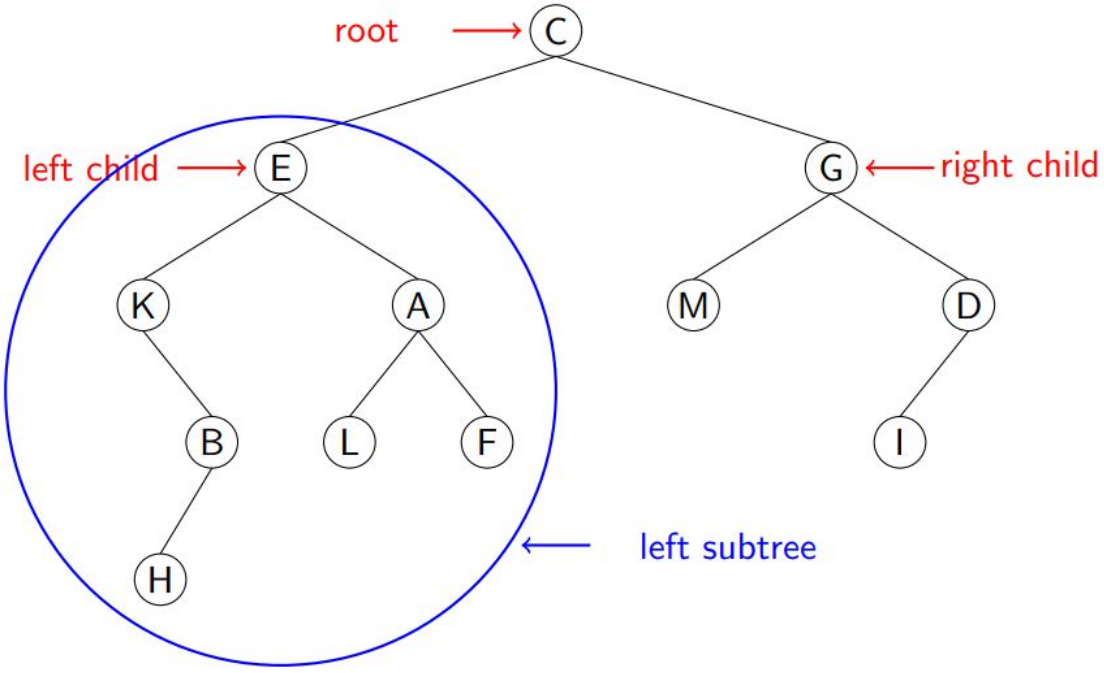
Abstract data type `BinTree`

OBJECTS: binary trees.

A binary tree is a node collection having the properties:

- any node has 0, 1 or 2 successors (children).
- any node except one — the root — has a single predecessor (parent).
- the root has no predecessors.
- the children are ordered: left child, right child (if a node has single child, it has to be specified which one);

Binary tree: example



Binary trees: properties

Notation:

- n – number of nodes.
- n_e – number of external nodes. (LEAFS)
- n_i – number of internal nodes.
- h – tree height.

$$h + 1 \leq n \leq 2^{h+1} - 1; \qquad \log_2(n+1) - 1 \leq h \leq n - 1$$

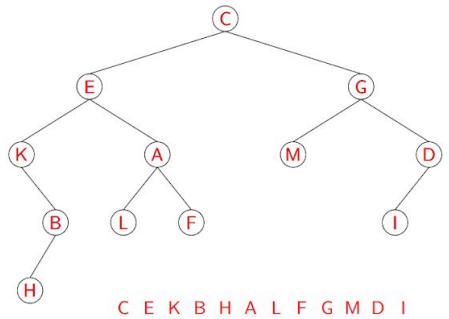
$$1 \leq n_e \leq 2^h; \qquad h \leq n_i \leq 2^h - 1$$

BinTree – preorder traversal

`preorder()`

- input:
  - a binary tree `t`;
  - a procedure `visit()`.
- output:
  - binary tree `t`, with the nodes processed by `visit()` in the following order
    - (R) – root
    - (S) – left subtree
    - (D) – right subtree

Preorder traversal - example



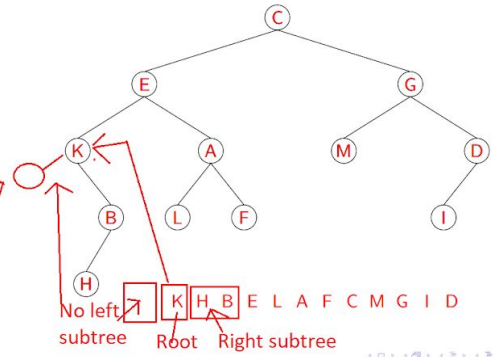
C E K B H A L F G M D I

BinTree – inorder traversal

`inorder()`

- input:
  - a binary tree `t`;
  - a procedure `visit()`.
- output:
  - binary tree `t` with nodes processed by `viziteaza()` in the following order
    - (S) – left subtree
    - (R) – root
    - (D) – right subtree

Inorder traversal – example



BinTree – postorder traversal

`postorder()`

- input:
  - a binary tree `t`;
  - a procedure `visit()`.
- output:
  - binary tree `t` with nodes processed by `visit()` in the following order
    - (S) – left subtree
    - (D) – right subtree
    - (R) – root

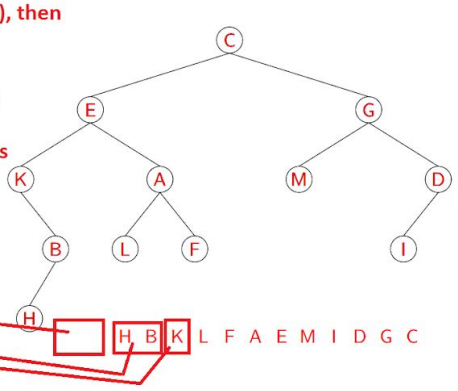
Note it goes first left (which does not exist), then to the right until a leaf then up so:

Nothing(no left), H then B then the root, K

Then it's finished the first left subtree, goes right,

L then F (right and left) then leaf A

Postorder traversal – example

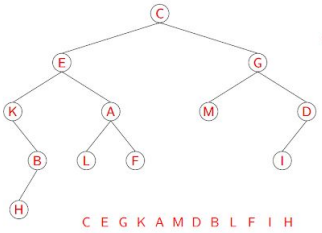


BinTree – BFS traversal

`BFS()` – Breadth-First Search

- input:
  - a binary tree `t`;
  - a procedure `visit()`.
- output:
  - binary tree `t` with the nodes processed by `visit()` in BFS order (level by level).

BFS traversal – example



1 C
2 ~~K~~ E G — C
3 ~~E~~ G K A — C E
4 ~~G~~ K A M D — C E G
5 K A M B B — C E G K

Content

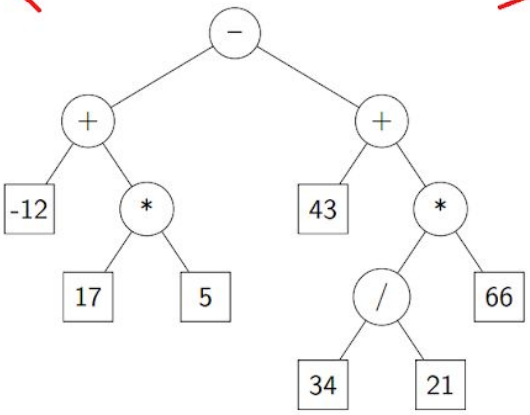
Trees

Binary tree (`BinTree`)

Application: integer expression representation as trees

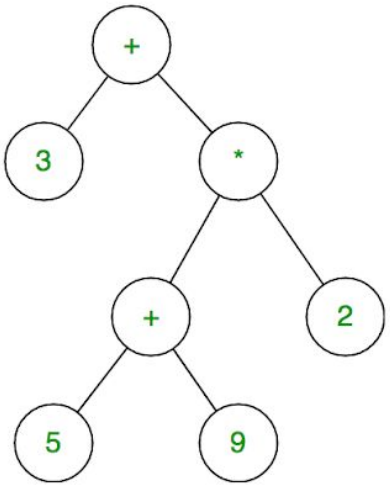
Application: integer expression

$$-12 + 17 * 5 - (43 + 34/21 * 66)$$



- Integer expressions
  - definition;
  - examples.
- Tree representation of integer expressions
  - definition similarities;
  - expression associated tree;
  - prefix, infix and postfix notation and tree traversal.

3 + ((5+9)*2)



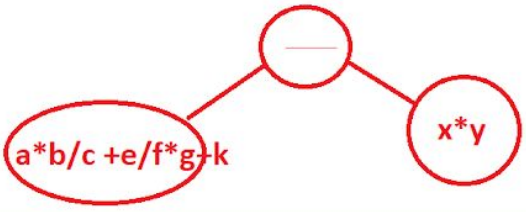
Expresia: **Minus is last, hence it is root**

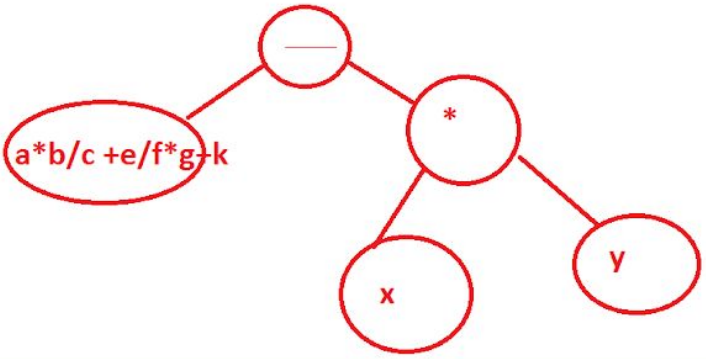
a*b/c + e/f * g + k - x*y

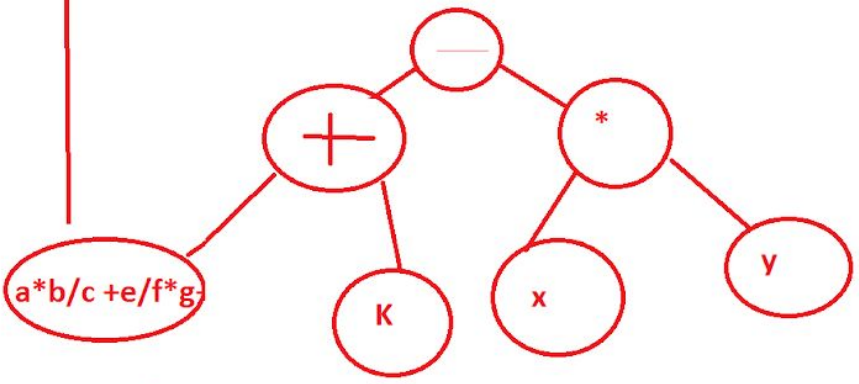
1. From left to right find the last operator

2. Select it as root

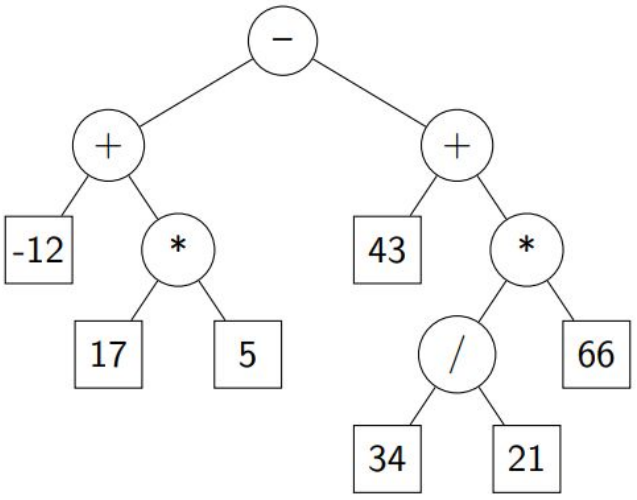
Don't forget about opperator precedence







https://youtu.be/2Z6g3kNymd0

Postfix and prefix notations

- postfix notation is given by the postorder traversal
  $-12, 17, 5, *, +, 43, 34, 21, /, 66, *, +, -$
- prefix notation is given by the preorder traversal
  $-, +, -12, *, 17, 5, +, 43, *, /, 34, 21, 66$